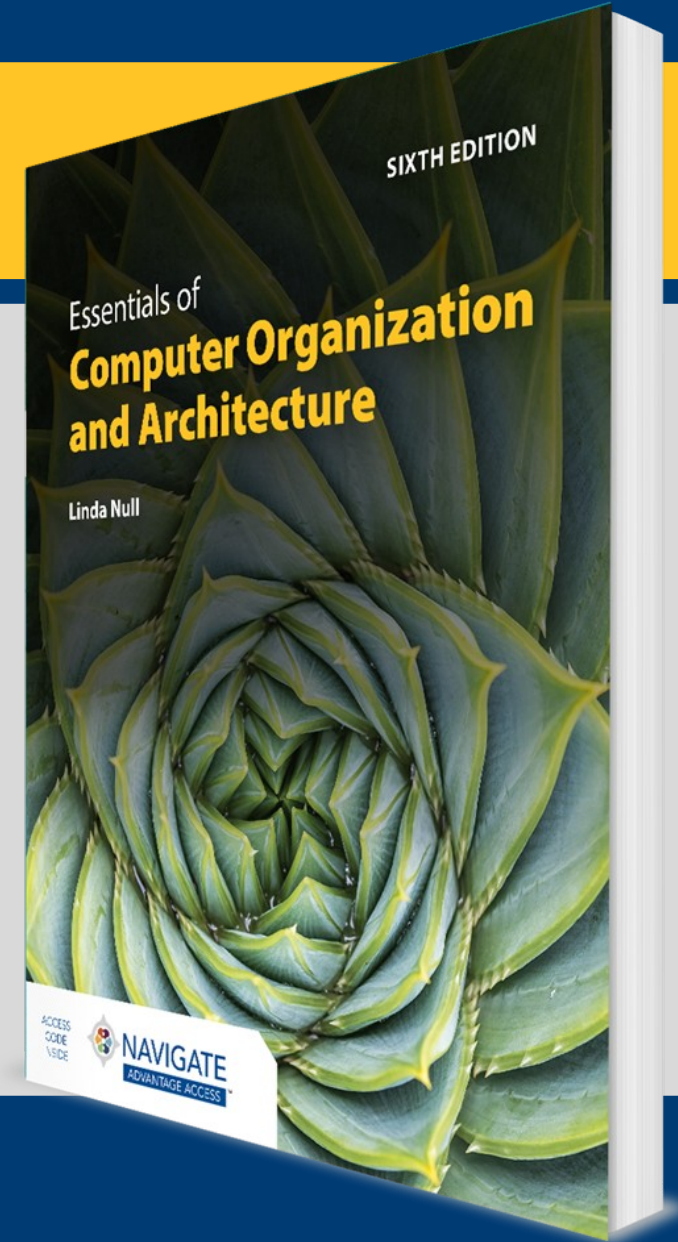


CHAPTER 6

Memory



Objectives

- Master the concepts of hierarchical memory organization.
- Understand how each level of memory contributes to system performance, and how the performance is measured.
- Master the concepts behind cache memory, virtual memory, memory segmentation, paging, and address translation.

6.1 Introduction

- Memory lies at the heart of the stored-program computer.
- In previous chapters, we studied the components from which memory is built and the ways in which memory is accessed by various ISAs.
- In this chapter, we focus on memory organization. A clear understanding of these ideas is essential for the analysis of system performance.

6.2 Types of Memory (1 of 2)

- There are two kinds of main memory: random access memory (RAM) and read-only memory (ROM).
- There are two types of RAM: dynamic RAM (DRAM) and static RAM (SRAM).
- DRAM consists of capacitors that slowly leak their charge over time. Thus, they must be refreshed every few milliseconds to prevent data loss.
- DRAM is “cheap” memory owing to its simple design.

6.2 Types of Memory (2 of 2)

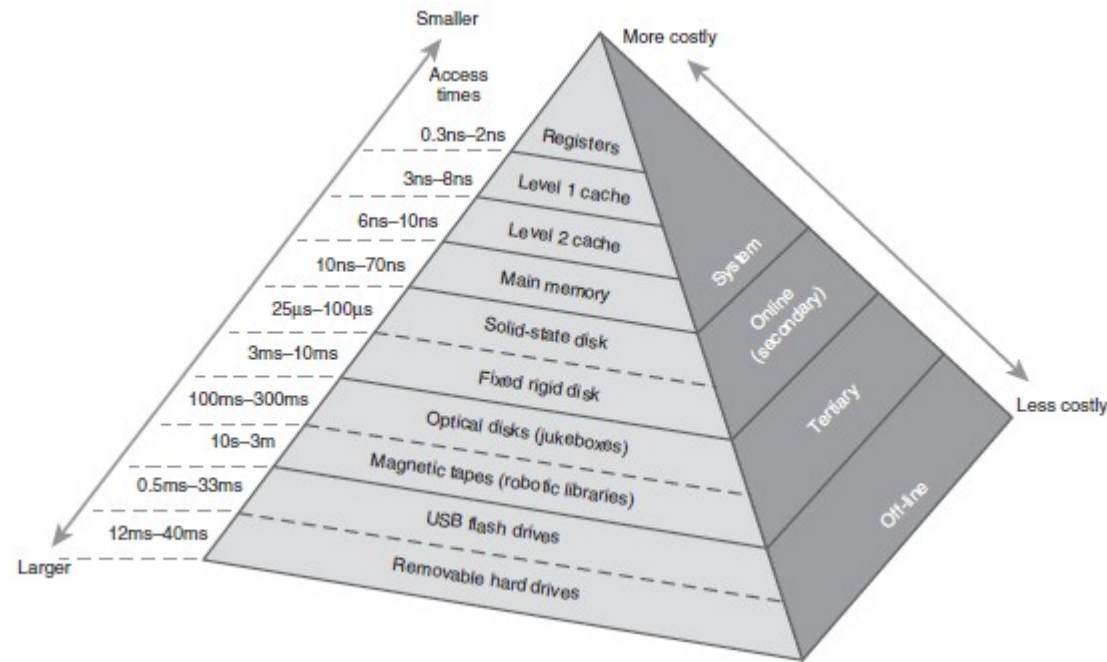
- SRAM consists of circuits similar to the D flip-flop that we studied in Chapter 3.
- SRAM is very fast memory, and it doesn't need to be refreshed like as DRAM does. It is used to build cache memory, which we will discuss in detail later.
- ROM also does not need to be refreshed, either. In fact, it needs very little charge to retain its memory.
- ROM is used to store permanent, or semi-permanent, data that persists even while the system is turned off.

6.3 The Memory Hierarchy (1 of 6)

- Generally speaking, faster memory is more expensive than slower memory.
- To provide the best performance at the lowest cost, memory is organized in a hierarchical fashion.
- Small, fast storage elements are kept in the CPU; larger, slower main memory is accessed through the data bus.
- Larger, (almost) permanent storage in the form of disk and tape drives is still further from the CPU.

6.3 The Memory Hierarchy (2 of 6)

- This storage organization can be thought of as a pyramid:



6.3 The Memory Hierarchy (3 of 6)

- We are most interested in the memory hierarchy that involves registers, cache, main memory, and virtual memory.
- Registers are storage locations available on the processor itself.
- Virtual memory is typically implemented using a hard drive; it extends the address space from RAM to the hard drive.
- Virtual memory provides more space: Cache memory provides speed.

6.3 The Memory Hierarchy (4 of 6)

- To access a particular piece of data, the CPU first sends a request to its nearest memory, usually cache.
- If the data is not in cache, then main memory is queried. If the data is not in main memory, then the request goes to disk.
- Once the data is located, then the data and a number of its nearby data elements are fetched into cache memory.

6.3 The Memory Hierarchy (5 of 6)

- This leads us to some definitions.
 - A hit is when data is found at a given memory level.
 - A miss is when it is not found.
 - The hit rate is the percentage of time data is found at a given memory level.
 - The miss rate is the percentage of time it is not.
 - Miss rate = $1 - \text{hit rate}$.
 - The hit time is the time required to access data at a given memory level.
 - The miss penalty is the time required to process a miss, including the time that it takes to replace a block of memory plus the time it takes to deliver the data to the processor.

6.3 The Memory Hierarchy (6 of 6)

- An entire block of data is copied after a hit because the principle of locality tells us that once a byte is accessed, it's likely that a nearby data element will be needed soon.
- There are three forms of locality:
 - Temporal locality: Recently accessed data elements tend to be accessed again.
 - Spatial locality: Accesses tend to cluster.
 - Sequential locality: Instructions tend to be accessed sequentially.

6.4 Cache Memory (1 of 45)

- The purpose of cache memory is to speed up accesses by storing recently used data closer to the CPU, instead of storing it in main memory.
- Although cache is much smaller than main memory, its access time is a fraction of that of main memory.
- Unlike main memory, which is accessed by address, cache is typically accessed by content; hence, it is often called content addressable memory.
- Because of this, a single large cache memory isn't always desirable—it takes longer to search.

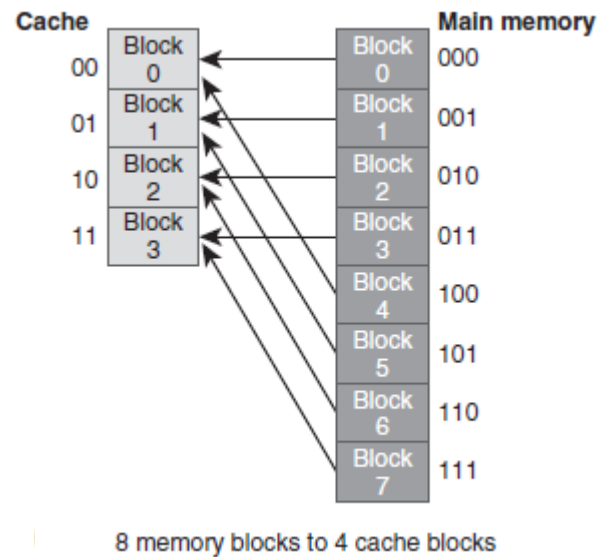
6.4 Cache Memory (2 of 45)

- The simplest cache mapping scheme is a direct-mapped cache.
- In a direct-mapped cache consisting of N blocks of cache, block X of main memory maps to cache block $Y = X \bmod N$.
- Thus, if we have 10 blocks of cache, block 7 of the cache may hold blocks 7, 17, 27, 37, . . . of main memory.

The next slide illustrates this mapping concept.

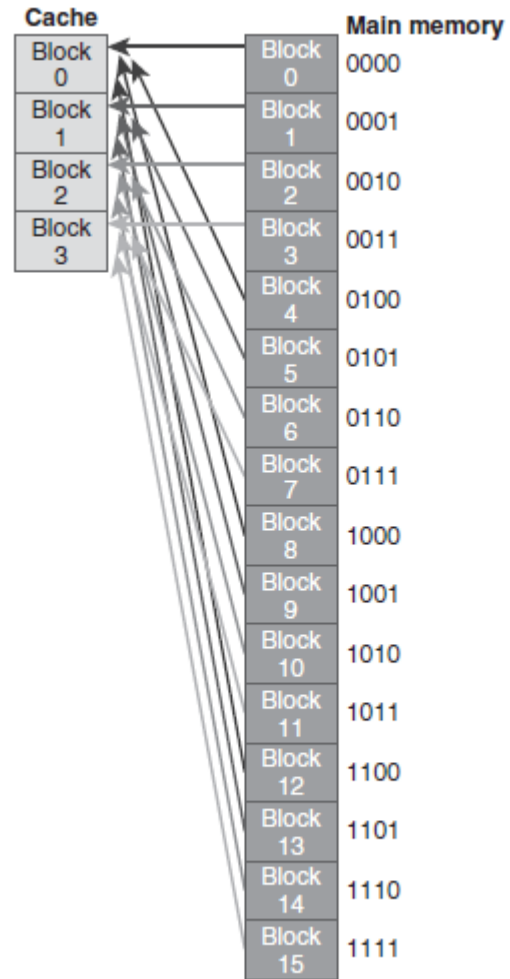
6.4 Cache Memory (3 of 45)

- With a direct-mapped cache consisting of 4 blocks of cache, block X of main memory maps to cache block $Y = X \bmod 4$.



6.4 Cache Memory (4 of 45)

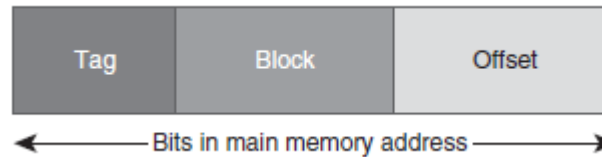
- A larger example



16 memory blocks to 4 cache blocks

6.4 Cache Memory (5 of 45)

- To perform direct mapping, the binary main memory address is partitioned into the fields shown below.
 - The offset field uniquely identifies an address within a specific block.
 - The block field selects a unique block of cache.
 - The tag field is whatever is left over.



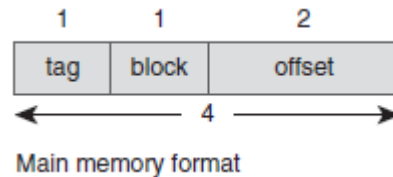
- The sizes of these fields are determined by characteristics of both memory and cache.

6.4 Cache Memory (6 of 45)

- Example 6.1: Consider a byte-addressable main memory consisting of 4 blocks, and a cache with 2 blocks, where each block is 4 bytes.
- This means Blockblocks 0 and 2 of main memory map to Blockblock 0 of cache, and Blocksblocks 1 and 3 of main memory map to Blockblock 1 of cache.
- Using the tag, block, and offset fields, we can see how main memory maps to cache as follows.

6.4 Cache Memory (7 of 45)

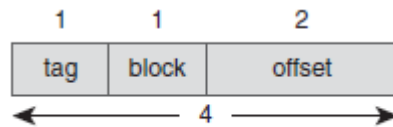
- Example 6.1: Cont'd. Consider a byte-addressable main memory consisting of 4 blocks, and a cache with 2 blocks, where each block is 4 bytes.
 - First, we need to determine the address format for mapping. Each block is 4 bytes, so the offset field must contain 2 bits; there are 2 blocks in cache, so the block field must contain 1 bit; this leaves 1 bit for the tag (as a main memory address has 4 bits because there are a total of $2^4 = 16$ bytes).



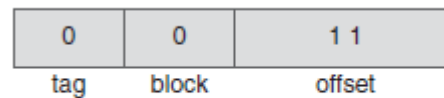
6.4 Cache Memory (8 of 45)

■ Example 6.1: Cont'd.

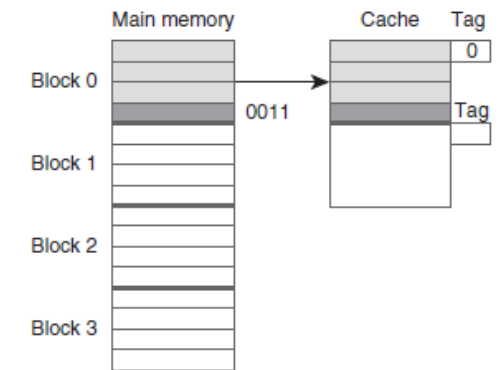
- Suppose we need to access main memory address 0x3 (0011 in binary). If we partition 0x3 using the address format from the top figure, we get the middle figure.
- Thus, the main memory address 0x3 maps to cache block 0.
- The bottom figure shows this mapping, along with the tag that is also stored with the data.



Main memory format



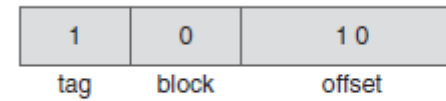
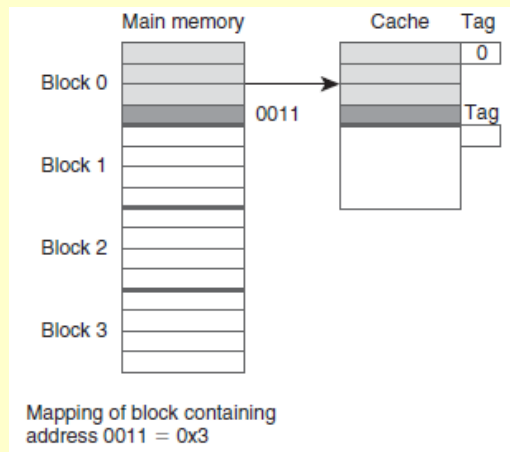
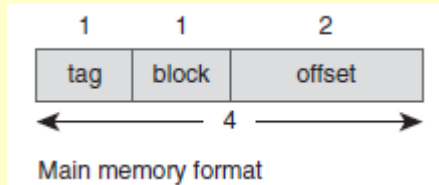
The address 0011 partitioned into fields



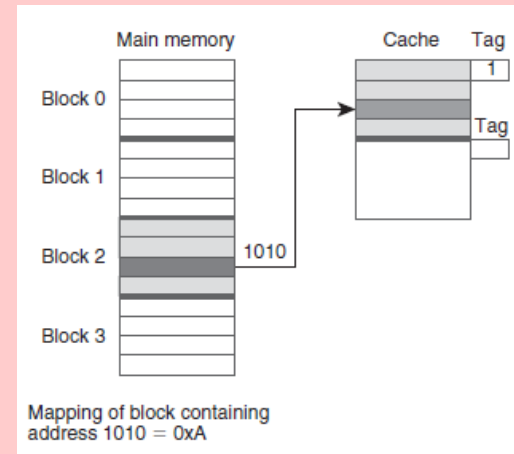
Mapping of block containing address 0011 = 0x3

The next slide illustrates another mapping for address 0xA or 1010 in binary.

6.4 Cache Memory (9 of 45)

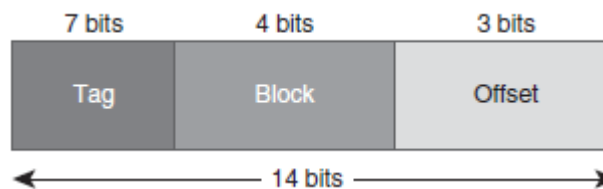


The address 1010 partitioned into fields



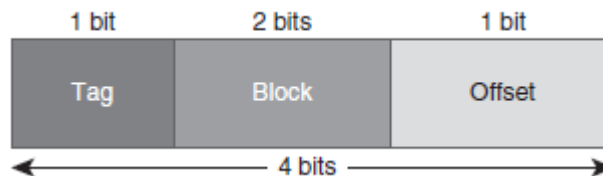
6.4 Cache Memory (10 of 45)

- Example 6.2: Assume a byte-addressable memory consists of 214 bytes, cache has 16 blocks, and each block has 8 bytes.
 - The number of memory blocks are: $\frac{2^{14}}{2^3} = 2^{11}$
 - Each main memory address requires 14 bits. Of this 14-bit address field, the rightmost 3 bits reflect the offset field.
 - We need 4 bits to select a specific block in cache, so the block field consists of the middle 4 bits.
 - The remaining 7 bits make up the tag field.



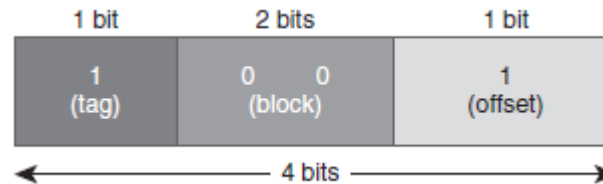
6.4 Cache Memory (11 of 45)

- Example 6.3: Assume a byte-addressable memory consisting of 16 bytes divided into 8 blocks. Cache contains 4 blocks. We know:
 - A memory address has 4 bits.
 - The 4-bit memory address is divided into the fields below.



6.4 Cache Memory (12 of 45)

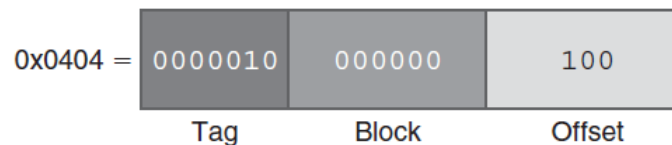
- Example 6.3: Cont'd. The mapping for memory references is shown below:



Main Memory	Maps To	Cache
(000) Block 0 (addresses 0x0, 0x1)	→	Block 0 (00)
(001) Block 1 (addresses 0x2, 0x3)	→	Block 1 (01)
(010) Block 2 (addresses 0x4, 0x5)	→	Block 2 (10)
(011) Block 3 (addresses 0x6, 0x7)	→	Block 3 (11)
(100) Block 4 (addresses 0x8, 0x9)	→	Block 0 (00)
(101) Block 5 (addresses 0xA, 0xB)	→	Block 1 (01)
(110) Block 6 (addresses 0xC, 0xD)	→	Block 2 (10)
(111) Block 7 (addresses 0xE, 0xF)	→	Block 3 (11)

6.4 Cache Memory (13 of 45)

- Example 6.4: Consider 16-bit memory addresses and 64 blocks of cache where each block contains 8 bytes. We have:
 - 3 bits for the offset
 - 6 bits for the block
 - 7 bits for the tag
- A memory reference for 0x0404 maps as follows:



6.4 Cache Memory (14 of 45)

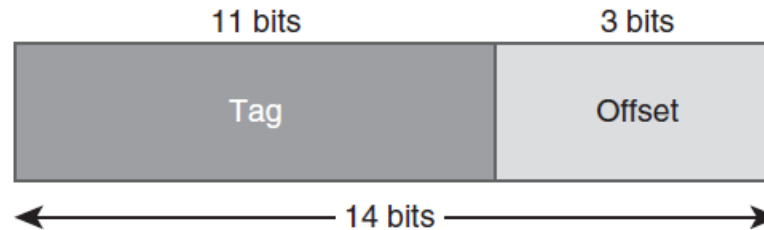
- In summary, direct-mapped cache maps main memory blocks in a modular fashion to cache blocks. The mapping depends on:
 - The number of bits in the main memory address (how many addresses exist in main memory).
 - The number of blocks that are in cache (which determines the size of the block field).
 - How many addresses (either bytes or words) are in a block (which determines the size of the offset field)?

6.4 Cache Memory (15 of 45)

- Suppose instead of placing memory blocks in specific cache locations based on memory address, we could allow a block to go anywhere in cache.
- In this way, cache would have to fill up before any blocks are evicted.
- This is how fully associative cache works.
- A memory address is partitioned into only two fields: the tag and the offset.

6.4 Cache Memory (16 of 45)

- Suppose, as before, we have 14-bit memory addresses and a cache with 16 blocks, each block of size 8. The field format of a memory reference is:



- When the cache is searched, all tags are searched in parallel to retrieve the data quickly.
- This requires special, costly hardware.

6.4 Cache Memory (17 of 45)

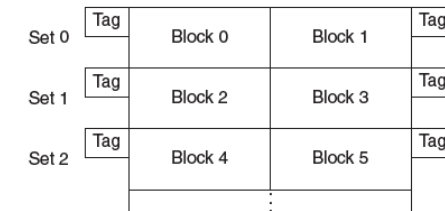
- You will recall that a direct-mapped cache evicts a block whenever another memory reference needs that block.
- With a fully associative cache, we have no such mapping, thus we must devise an algorithm to determine which block to evict from the cache.
- The block that is evicted is the victim block.
- There are a number of ways to pick a victim, we will discuss them shortly.

6.4 Cache Memory (18 of 45)

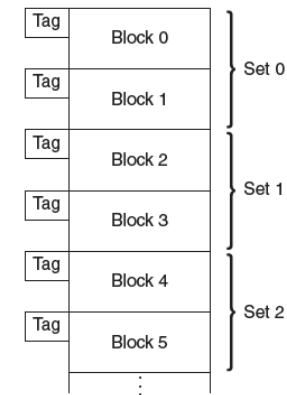
- Set-associative cache combines the ideas of direct-mapped cache and fully associative cache.
- An N-way set-associative cache mapping is like a direct-mapped cache in that a memory reference maps to a particular location in cache.
- Unlike direct-mapped cache, a memory reference maps to a set of several cache blocks, similar to the way in which fully associative cache works.
- Instead of mapping anywhere in the entire cache, a memory reference can map only to the subset of cache slots.

6.4 Cache Memory (19 of 45)

- The number of cache blocks per set in a set-associative cache varies according to overall system design.
 - For example, a 2-way set associative cache can be conceptualized as shown in the schematic to the right.
 - Each set contains two different memory blocks.



A Logical view of 2-way set associative cache



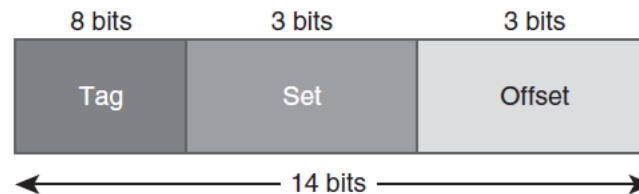
B Linear view of 2-way set associative cache

6.4 Cache Memory (20 of 45)

- In set-associative cache mapping, a memory reference is divided into three fields: tag, set, and offset.
- As with direct-mapped cache, the offset field chooses the byte within the cache block, and the tag field uniquely identifies the memory address.
- The set field determines the set to which the memory block maps.

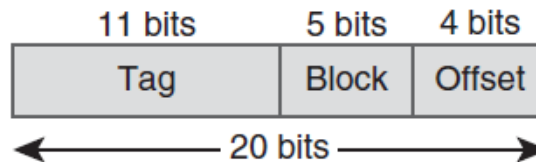
6.4 Cache Memory (21 of 45)

- Example 6.5: Suppose we are using 2-way set-associative mapping with a byte-addressable main memory of 2^{14} bytes and a cache with 16 blocks, where each block contains 8 bytes.
 - Cache has a total of 16 blocks, and each set has 2 blocks, then there are 8 sets in cache.
 - Thus, the set field is 3 bits, the offset field is 3 bits, and the tag field is 8 bits.



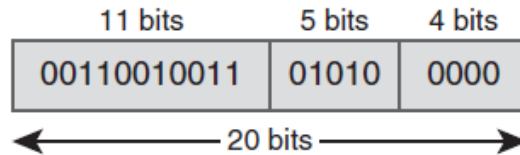
6.4 Cache Memory (22 of 45)

- Example 6.6: Suppose a byte-addressable memory contains 1MB and cache consists of 32 blocks, where each block contains 16 bytes. Using direct mapping, fully associative mapping, and a 4-way set-associative mapping, determine where the main memory address 0x326A0 maps to in cache.
 - First note that a main memory address has 20 bits. The main memory address for a direct-mapped cache is shown below.



6.4 Cache Memory (23 of 45)

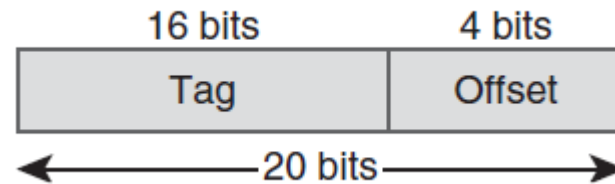
- Example 6.6:
 - If we represent our main memory address 0x326A0 in binary and place the bits into the format, we get:



- So, this address maps to cache block 01010 (or block 10).

6.4 Cache Memory (24 of 45)

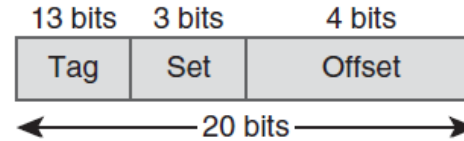
- Example 6.6: Cont'd.
 - If we are using fully associative cache, we have:



- But because it is fully associative, it could map anywhere.

6.4 Cache Memory (25 of 45)

- Example 6.6: Cont'd.
 - If we are using 4-way set-associative cache, we have:



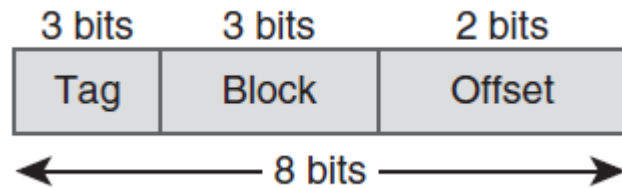
- If we divide the main m

13 bits	3 bits	4 bits
0011001001101	010	0000

 these fields, we get:

6.4 Cache Memory (26 of 45)

- Example 6.7: A byte-addressable computer with an 8-block cache of 4 bytes each, trace memory accesses: 0x01, 0x04, 0x09, 0x05, 0x14, 0x21, and 0x01 for each mapping approach.
- The address format for the direct-mapped cache is:



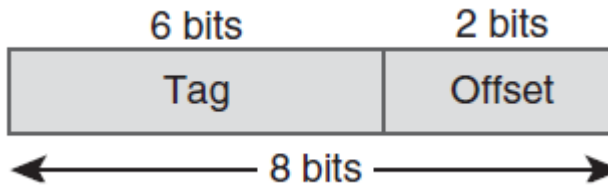
Our trace is on the next slide.

6.4 Cache Memory (27 of 45)

Address Reference	Binary Address (divided into fields)	Hit or Miss	Comments
0x01	000 000 01	Miss	If we check cache block 000 for the tag 000, we find that it is not there. So we copy the data from addresses 0x00, 0x01, 0x02, and 0x03 into cache block 0 and store the tag 000 for that block.
0x04	000 001 00	Miss	We check cache block 001 for the tag 000, and on finding it missing, we copy the data from addresses 0x04, 0x05, 0x06, and 0x07 into cache block 1 and store the tag 000 for that block.
0x09	000 010 01	Miss	A check of cache block 010 (2) for the tag 000 reveals a miss, so we copy the data from addresses 0x08, 0x09, 0x0A, and 0x0B into cache block 2 and store the tag 000 for that block.
0x05	000 001 01	Hit	We check cache block 001 for the tag 000, and we find it. We then use the offset value 01 to get the exact byte we need.
0x14	000 101 00	Miss	We check cache block 101 (5) for the tag 000, but it is not present. We copy addresses 0x14, 0x15, 0x16, and 0x17 to cache block 5 and store the tag 000 with that block.
0x21	001 000 01	Miss	We check cache block 000 for the tag 001; we find tag 000 (which means this is not the correct block), so we overwrite the existing contents of this cache block by copying the data from addresses 0x20, 0x21, 0x22, and 0x23 into cache block 0 and storing the tag 001.
0x01	000 000 01	Miss	Although we have already fetched the block that contains address 0x01 once, it was overwritten when we fetched the block containing address 0x21 (if we look at block 0 in cache, we can see that its tag is 001, not 000). Therefore, we must overwrite the contents of block 0 in cache with the data from addresses 0x00, 0x01, 0x02, and 0x03, and store a tag of 000.

6.4 Cache Memory (28 of 45)

- Example 6.7: Cont'd. A byte-addressable computer with an 8-block cache of 4 bytes each, trace memory accesses: 0x01, 0x04, 0x09, 0x05, 0x14, 0x21, and 0x01 for each mapping approach.
- The address format for fully associative cache is:



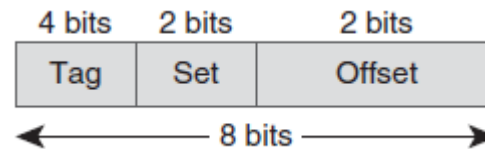
Our trace is on the next slide.

6.4 Cache Memory (29 of 45)

Address Reference	Binary Address (divided into fields)	Hit or Miss	Comments
0x01	000000 01	Miss	We search all of cache for the tag 000000, and we don't find it. So we copy the data from addresses 0x00, 0x01, 0x02, and 0x03 into cache block 0 and store the tag 000000 for that block.
0x04	000001 00	Miss	We search all of cache for the tag 000001, and on finding it missing, we copy the data from addresses 0x04, 0x05, 0x06, and 0x07 into cache block 1 and store the tag 000001 for that block.
0x09	000010 01	Miss	We don't find the tag 000010 in cache, so we copy the data from addresses 0x08, 0x09, 0x0A, and 0x0B into cache block 2 and store the tag 000010 for that block.
0x05	000001 01	Hit	We search all of cache for the tag 000001, and we find it stored with cache block 1. We then use the offset value 01 to get the exact byte we need.
0x14	000101 00	Miss	We search all of cache for the tag 000101, but it is not present. We copy addresses 0x14, 0x15, 0x16, and 0x17 to cache block 3 and store the tag 000101 with that block.
0x21	001000 01	Miss	We search all of cache for the tag 001000; we don't find it, so we copy the data from addresses 0x20, 0x21, 0x22, and 0x23 into cache block 4 and store the tag 001000.
0x01	000000 01	Hit	We search cache for the tag 000000 and find it with cache block 0. We use the offset of 1 to find the data we want.

6.4 Cache Memory (30 of 45)

- EXAMPLE 6.7: Cont'd. A byte-addressable computer with an 8-block cache of 4 bytes each, trace memory accesses: 0x01, 0x04, 0x09, 0x05, 0x14, 0x21, and 0x01 for each mapping approach.
- The address format for 2-way set-associative cache is:



Our trace is on the next slide.

6.4 Cache Memory (31 of 45)

Address Reference	Binary Address (divided into fields)	Hit or Miss	Comments
0x01	0000 00 01	Miss	We search in set 0 of cache for a block with the tag 0000, and we find it is not there. So we copy the data from addresses 0x00, 0x01, 0x02, and 0x03 into set 0 (so now set 0 has one used block and one free block) and store the tag 0000 for that block. It does not matter which set we use; for consistency, we put the data in the first set.
0x04	0000 01 00	Miss	We search set 1 for a block with the tag 0000, and on finding it missing, we copy the data from addresses 0x04, 0x05, 0x06, and 0x07 into set and store the tag 0000 for that block.
0x09	0000 10 01	Miss	We search set 2 (10) for a block with the tag 0000, but we don't find one, so we copy the data from addresses 0x08, 0x09, 0x0A, and 0x0B into set 2 and store the tag 0000 for that block.
0x05	0000 01 01	Hit	We search set 1 for a block with the tag 0000, and we find it. We then use the offset value 01 within that block to get the exact byte we need.
0x14	0001 01 00	Miss	We search set 1 for a block with the tag 0001, but it is not present. We copy addresses 0x14, 0x15, 0x16, and 0x17 to set 1 and store the tag 0001 with that block. Note that set 1 is now full.
0x21	0010 00 01	Miss	We search cache set 0 for a block with the tag 0010; we don't find it, so we copy the data from addresses 0x20, 0x21, 0x22, and 0x23 into set 0 and store the tag 0010. Note that set 0 is now full.
0x01	0000 00 01	Hit	We search cache set 0 for a block with the tag 0000, and we find it. We use the offset of 1 within that block to find the data we want.

6.4 Cache Memory (32 of 45)

- With fully associative and set-associative cache, a replacement policy is invoked when it becomes necessary to evict a block from cache.
- An optimal replacement policy would be able to look into the future to see which blocks won't be needed for the longest period of time.
- Although it is impossible to implement an optimal replacement algorithm, it is instructive to use it as a benchmark for assessing the efficiency of any other scheme we come up with.

6.4 Cache Memory (33 of 45)

- The replacement policy that we choose depends upon the locality that we are trying to optimize—usually, we are interested in temporal locality.
- A least recently used (LRU) algorithm keeps track of the last time that a block was assessed and evicts the block that has been unused for the longest period of time.
- The disadvantage of this approach is its complexity: LRU has to maintain an access history for each block, which ultimately slows down the cache.

6.4 Cache Memory (34 of 45)

- First-in, first-out (FIFO) is a popular cache replacement policy.
- In FIFO, the block that has been in the cache the longest, regardless of when it was last used.
- A random replacement policy does what its name implies: It picks a block at random and replaces it with a new block.
- Random replacement can certainly evict a block that will be needed often or needed soon, but it never thrashes.

6.4 Cache Memory (35 of 45)

- The performance of hierarchical memory is measured by its effective access time (EAT).
- EAT is a weighted average that takes into account the hit ratio and relative access times of successive levels of memory.
- The EAT for a two-level memory is given by:

$$\text{EAT} = H \times \text{AccessC} + (1 - H) \times \text{AccessMM}$$

where H is the cache hit rate, and AccessC and AccessMM are the access times for cache and main memory, respectively.

6.4 Cache Memory (36 of 45)

- For example, consider a system with a main memory access time of 200ns supported by a cache having a 10ns access time and a hit rate of 99%.
- Suppose access to cache and main memory occurs concurrently (the accesses overlap).
- The EAT is:

$$0.99(10\text{ns}) + 0.01(200\text{ns}) = 9.9\text{ns} + 2\text{ns} = 11\text{ns}$$

6.4 Cache Memory (37 of 45)

- For example, consider a system with a main memory access time of 200ns supported by a cache having a 10ns access time and a hit rate of 99%.
- If the accesses do not overlap, the EAT is:
$$0.99(10\text{ns}) + 0.01(10\text{ns} + 200\text{ns})$$
$$= 9.9\text{ns} + 2.01\text{ns} = 12\text{ns}$$
- This equation for determining the effective access time can be extended to any number of memory levels, as we will see in later sections.

6.4 Cache Memory (38 of 45)

- Caching depends upon programs exhibiting good locality.
 - Some object-oriented programs have poor locality owing to their complex, dynamic structures.
 - Arrays stored in column-major rather than row-major order can be problematic for certain cache organizations.
- With poor locality, caching can actually cause performance degradation rather than performance improvement.

6.4 Cache Memory (39 of 45)

- Cache replacement policies must take into account dirty blocks, those blocks that have been updated while they were in the cache.
- Dirty blocks must be written back to memory. A write policy determines how this will be done.
- There are two types of write policies: write through and write back.
- Write through updates cache and main memory simultaneously on every write.
- Write back (also called copyback) updates memory only when the block is selected for replacement.

6.4 Cache Memory (40 of 45)

- The disadvantage of write through is that memory must be updated with each cache write, which slows down the access time on updates. This slowdown is usually negligible, because the majority of accesses tend to be reads, not writes.
- The advantage of write back is that memory traffic is minimized, but its disadvantage is that memory does not always agree with the value in cache, causing problems in systems with many concurrent users.

6.4 Cache Memory (41 of 45)

- The cache we have been discussing is called a unified or integrated cache where both instructions and data are cached.
- Many modern systems employ separate caches for data and instructions.
 - This is called a Harvard cache.
- The separation of data from instructions provides better locality, at the cost of greater complexity.
 - Simply making the cache larger provides about the same performance improvement without the complexity.

6.4 Cache Memory (42 of 45)

- Cache performance can also be improved by adding a small associative cache to hold blocks that have been evicted recently.
 - This is called a victim cache.
- A trace cache is a variant of an instruction cache that holds decoded instructions for program branches, giving the illusion that noncontiguous instructions are really contiguous.

6.4 Cache Memory (43 of 45)

- Most of today's small systems employ multilevel cache hierarchies.
- The levels of cache form their own small memory hierarchy.
- Level 1 cache (8KB to 64KB) is situated on the processor itself.
 - Access time is typically about 4ns.
- Level 2 cache (64KB to 2MB) may be on the motherboard, or on an expansion card.
 - Access time is usually around 15–20ns.

6.4 Cache Memory (44 of 45)

- In systems that employ three levels of cache, the Level 2 cache is placed on the same die as the CPU (reducing access time to about 10ns).
- Accordingly, the Level 3 cache (2MB to 256MB) refers to cache that is situated between the processor and main memory.
- Once the number of cache levels is determined, the next thing to consider is whether data (or instructions) can exist in more than one cache level.

6.4 Cache Memory (45 of 45)

- If the cache system used an inclusive cache, the same data may be present at multiple levels of cache.
- Strictly inclusive caches guarantee that all data in a smaller cache also exists at the next higher level.
- Exclusive caches permit only one copy of the data.
- The tradeoffs in choosing one over the other involve weighing the variables of access time, memory size, and circuit complexity.

6.5 Virtual Memory (1 of 26)

- Cache memory enhances performance by providing faster memory access speed.
- Virtual memory enhances performance by providing greater memory capacity, without the expense of adding main memory.
- Instead, a portion of a disk drive serves as an extension of main memory.
- If a system uses paging, virtual memory partitions main memory into individually managed page frames, that are written (or paged) to disk when they are not immediately needed.

6.5 Virtual Memory (2 of 26)

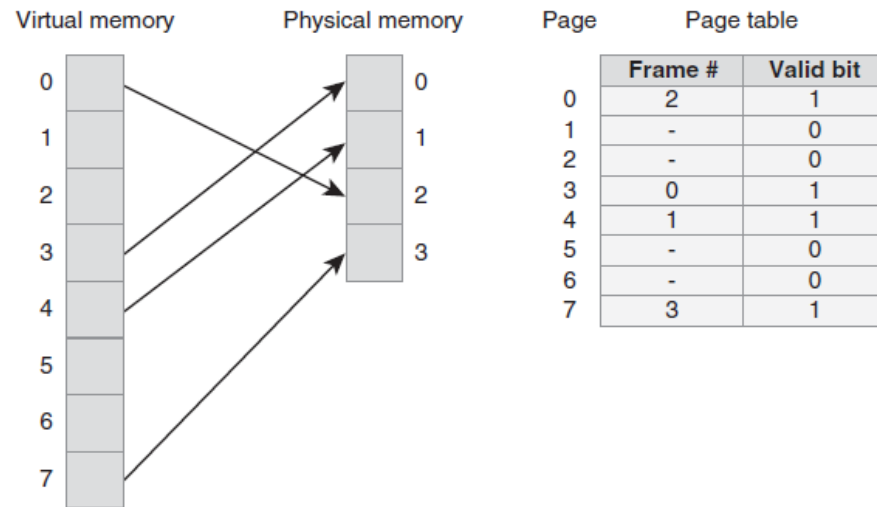
- A physical address is the actual memory address of physical memory.
- Programs create virtual addresses that are mapped to physical addresses by the memory manager.
- Page faults occur when a logical address requires that a page be brought in from disk.
- Memory fragmentation occurs when the paging process results in the creation of small, unusable clusters of memory addresses.

6.5 Virtual Memory (3 of 26)

- Main memory and virtual memory are divided into equal-sized pages.
- The entire address space required by a process need not be in memory at once. Some parts can be on disk, while others are in main memory.
- Further, the pages allocated to a process do not need to be stored contiguously—either on disk or in memory.
- In this way, only the needed pages are in memory at any time, the unnecessary pages are in slower disk storage.

6.5 Virtual Memory (4 of 26)

- Information concerning the location of each page, whether on disk or in memory, is maintained in a data structure called a page table (shown below).
- There is one page table for each active process.



6.5 Virtual Memory (5 of 26)

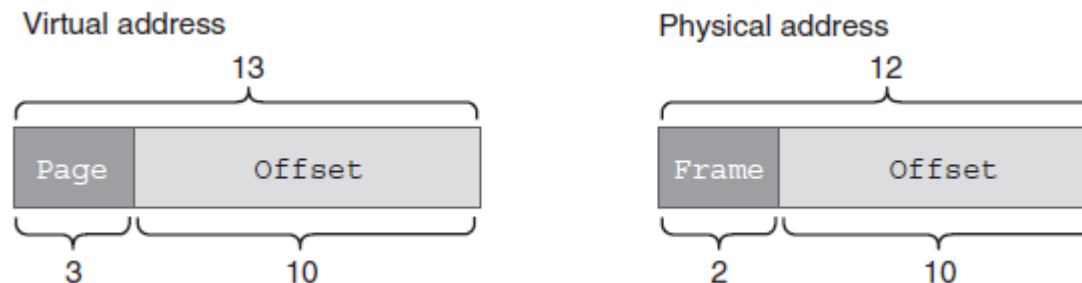
- When a process generates a virtual address, the operating system translates it into a physical memory address.
- To accomplish this, the virtual address is divided into two fields: a page field, and an offset field.
- The page field determines the page location of the address, and the offset indicates the location of the address within the page.
- The logical page number is translated into a physical page frame through a lookup in the page table.

6.5 Virtual Memory (6 of 26)

- If the valid bit is zero in the page table entry for the logical address, this means that the page is not in memory and must be fetched from disk.
 - This is a page fault.
 - If necessary, a page is evicted from memory and is replaced by the page retrieved from disk, and the valid bit is set to 1.
- If the valid bit is 1, the virtual page number is replaced by the physical frame number.
- The data is then accessed by adding the offset to the physical frame number.

6.5 Virtual Memory (7 of 26)

- As an example, suppose a system has a virtual address space of 8K and a physical address space of 4K, and the system uses byte addressing.
 - We have $2^{13}/2^{10} = 2^3 = 8$ virtual pages.
- A virtual address has 13 bits (8K = 2^{13}) with 3 bits for the page field and 10 for the offset, because the page size is 1024.
- A physical memory address requires 12 bits, the first 2 bits for the page frame and the trailing 10 bits for the offset.



6.5 Virtual Memory (8 of 26)

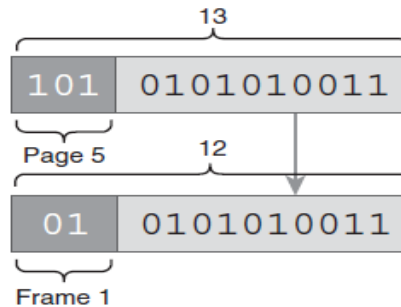
- Suppose we have the page table shown below.
- What happens when the CPU generates address 545910 = 10101010100112 = 0x1553?

Page table		
	Frame	Valid bit
Page 0	-	0
1	3	1
2	0	1
3	-	0
4	-	0
5	1	1
6	2	1
7	-	0

Addresses		
	Base 10	Base 16
Page 0 :	0 - 1023	0 - 3FF
1 :	1024 - 2047	400 - 7FF
2 :	2048 - 3071	800 - BFF
3 :	3072 - 4095	C00 - FFF
4 :	4096 - 5119	1000 - 13FF
5 :	5120 - 6143	1400 - 17FF
6 :	6144 - 7167	1800 - 1BFF
7 :	7168 - 8191	1C00 - 1FFF

6.5 Virtual Memory (9 of 26)

- What happens when the CPU generates address $545910 = 10101010100112 = 0x1553$?



- The high-order 3 bits of the virtual address, 101 (5_{10}), provide the page number in the page table.

6.5 Virtual Memory (10 of 26)

- The address 10101010100112 is converted to physical address 0101010100112 = 0x1363 because the page field 101 is replaced by frame number 01 through a lookup in the page table.

Page table		
	Frame	Valid bit
Page 0	-	0
1	3	1
2	0	1
3	-	0
4	-	0
5	1	1
6	2	1
7	-	0

Addresses		
	Base 10	Base 16
Page 0 :	0 - 1023	0 - 3FF
1 :	1024 - 2047	400 - 7FF
2 :	2048 - 3071	800 - BFF
3 :	3072 - 4095	C00 - FFF
4 :	4096 - 5119	1000 - 13FF
5 :	5120 - 6143	1400 - 17FF
6 :	6144 - 7167	1800 - 1BFF
7 :	7168 - 8191	1C00 - 1FFF

6.5 Virtual Memory (11 of 26)

- What happens when the CPU generates address 10000000001002?

Page table		
	Frame	Valid bit
Page 0	-	0
1	3	1
2	0	1
3	-	0
4	-	0
5	1	1
6	2	1
7	-	0

Addresses		
	Base 10	Base 16
Page 0	: 0 - 1023	0 - 3FF
1	: 1024 - 2047	400 - 7FF
2	: 2048 - 3071	800 - BFF
3	: 3072 - 4095	C00 - FFF
4	: 4096 - 5119	1000 - 13FF
5	: 5120 - 6143	1400 - 17FF
6	: 6144 - 7167	1800 - 1BFF
7	: 7168 - 8191	1C00 - 1FFF

6.5 Virtual Memory (12 of 26)

- We said earlier that effective access time (EAT) takes all levels of memory into consideration.
- Thus, virtual memory is also a factor in the calculation, and we also have to consider page table access time.
- Suppose a main memory access takes 200ns, the page fault rate is 1%, and it takes 10ms to load a page from disk. We have:
 - $EAT = 0.99(200ns + 200ns) + 0.01(10ms) = 100.396ns$

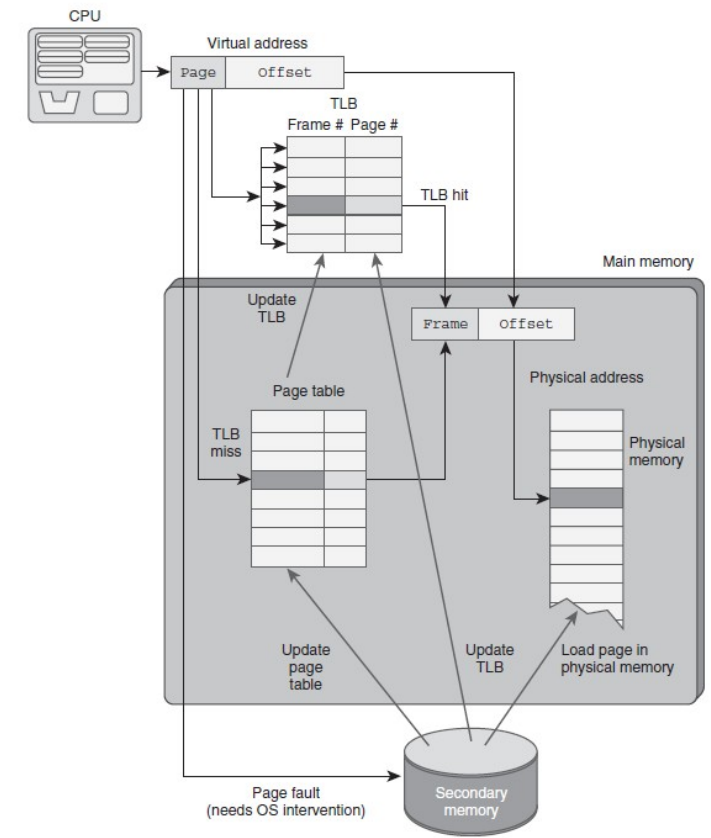
6.5 Virtual Memory (13 of 26)

- Even if we had no page faults, the EAT would be 400ns because memory is always read twice: First to access the page table, and second to load the page from memory.
- Because page tables are read constantly, it makes sense to keep them in a special cache called a translation look-aside buffer (TLB).
- TLBs are a special associative cache that stores the mapping of virtual pages to physical pages.

The next slide shows address lookup steps when a TLB is involved.

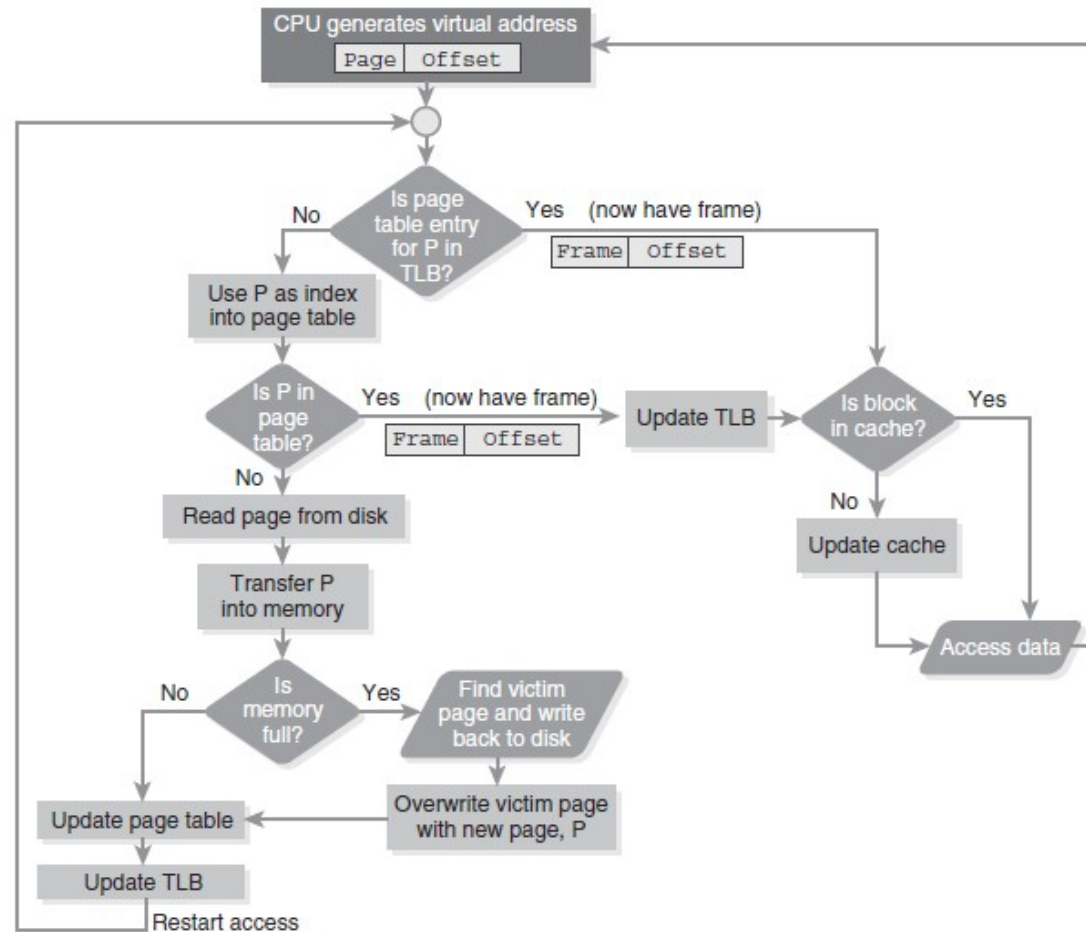
6.5 Virtual Memory (14 of 26)

- TLB lookup process
 - Extract the page number from the virtual address.
 - Extract the offset from the virtual address.
 - Search for the virtual page number in the TLB.
 - If the (virtual page #, page frame #) pair is found in the TLB, add the offset to the physical frame number and access the memory location.
 - If there is a TLB miss, go to the page table to get the necessary frame number. If the page is in memory, use the corresponding frame number and add the offset to yield the physical address.
 - If the page is not in main memory, generate a page fault and restart the access when the page fault is complete.



6.5 Virtual Memory (15 of 26)

Putting it all together: The TLB, Page Table, and Main Memory



6.5 Virtual Memory (16 of 26)

- Another approach to virtual memory is the use of segmentation.
- Instead of dividing memory into equal-sized pages, virtual address space is divided into variable-length segments, often under the control of the programmer.
- A segment is located through its entry in a segment table, which contains the segment's memory location and a bounds limit that indicates its size.
- After a page fault, the operating system searches for a location in memory large enough to hold the segment that is retrieved from disk.

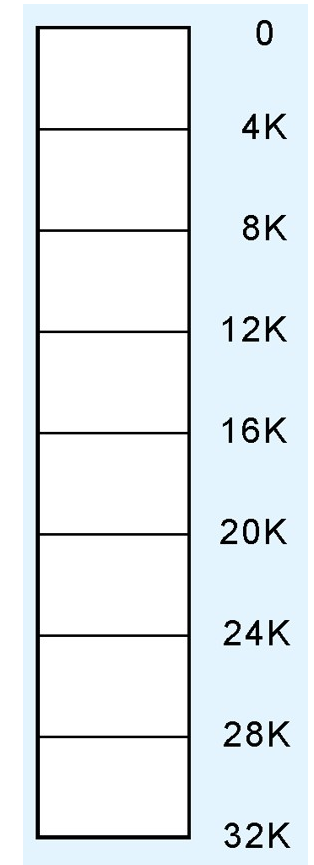
6.5 Virtual Memory (17 of 26)

- Both paging and segmentation can cause fragmentation.
- Paging is subject to internal fragmentation because a process may not need the entire range of addresses contained within the page. Thus, there may be many pages containing unused fragments of memory.
- Segmentation is subject to external fragmentation, which occurs when contiguous chunks of memory become broken up as segments are allocated and deallocated over time.

The next slides illustrate internal and external fragmentation.

6.5 Virtual Memory (18 of 26)

- Consider a small computer having 32K of memory.
- The 32K memory is divided into 8-page frames of 4K each.
- A schematic of this configuration is shown at on the right.
- The numbers at on the right are memory frame addresses.



6.5 Virtual Memory (19 of 26)

- Suppose there are four processes waiting to be loaded into the system with memory requirements as shown in the table.
- We observe that these processes require 31K of memory.

Process Name	Memory Needed
P1	8K
P2	10K
P3	9K
P4	4K

6.5 Virtual Memory (20 of 26)

- When the first three processes are loaded, memory looks like this:
- All of the frames are occupied by three of the processes.

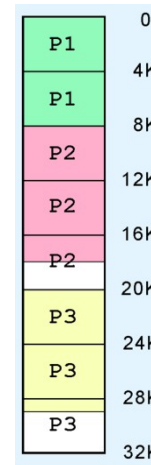
P1	8K
P2	10K
P3	9K
P4	4K

P1	0
P1	4K
P2	8K
P2	12K
P2	16K
	20K
P3	24K
P3	28K
P3	32K

6.5 Virtual Memory (21 of 26)

- Despite the fact that there are enough free bytes in memory to load the fourth process, P4 has to wait for one of the other three to terminate, because there are no unallocated frames.
- This is an example of internal fragmentation.

P1	8K
P2	10K
P3	9K
P4	4K



6.5 Virtual Memory (22 of 26)

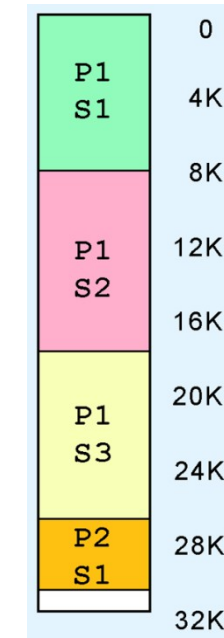
- Suppose that instead of frames, our 32K system uses segmentation.
- The memory segments of two processes is are shown in the table at on the right.
- The segments can be allocated anywhere in memory.

Process Name	Segment	Memory Needed
P1	S1	8K
	S2	10K
	S3	9K
P2	S1	4K
	S2	11K

6.5 Virtual Memory (23 of 26)

- All of the segments of P1 and one of the segments of P2 are loaded as shown at on the right.
- Segment S2 of process P2 requires 11K of memory, and there is only 1K free, so it waits.

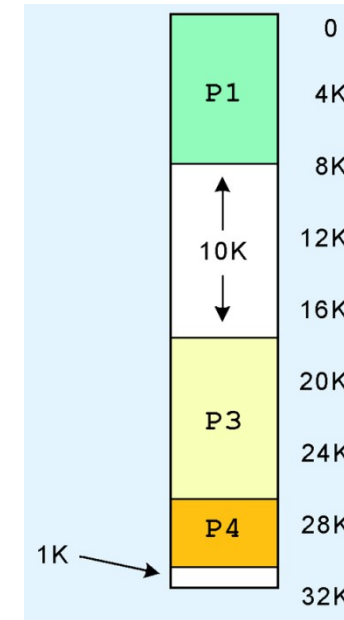
P1	S1	8K
	S2	10K
	S3	9K
P2	S1	4K
	S2	11K



6.5 Virtual Memory (24 of 26)

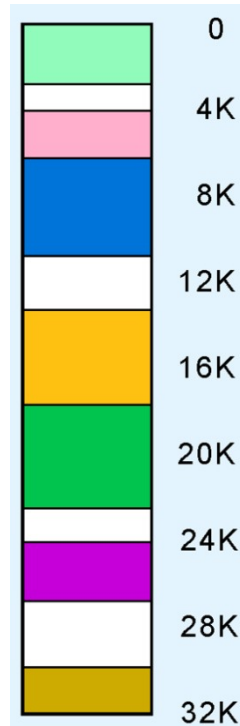
- Eventually, Segment 2 of Process 1 is no longer needed, so it is unloaded giving 11K of free memory.
- But Segment 2 of Process 2 cannot be loaded because the free memory is not contiguous.

P1	S1	8K
	S2	10K
	S3	9K
P2	S1	4K
	S2	11K



6.5 Virtual Memory (25 of 26)

- Over time, the problem gets worse, resulting in small unusable blocks scattered throughout physical memory.
- This is an example of external fragmentation.
- Eventually, this memory is recovered through compaction, and the process starts over.



6.5 Virtual Memory (26 of 26)

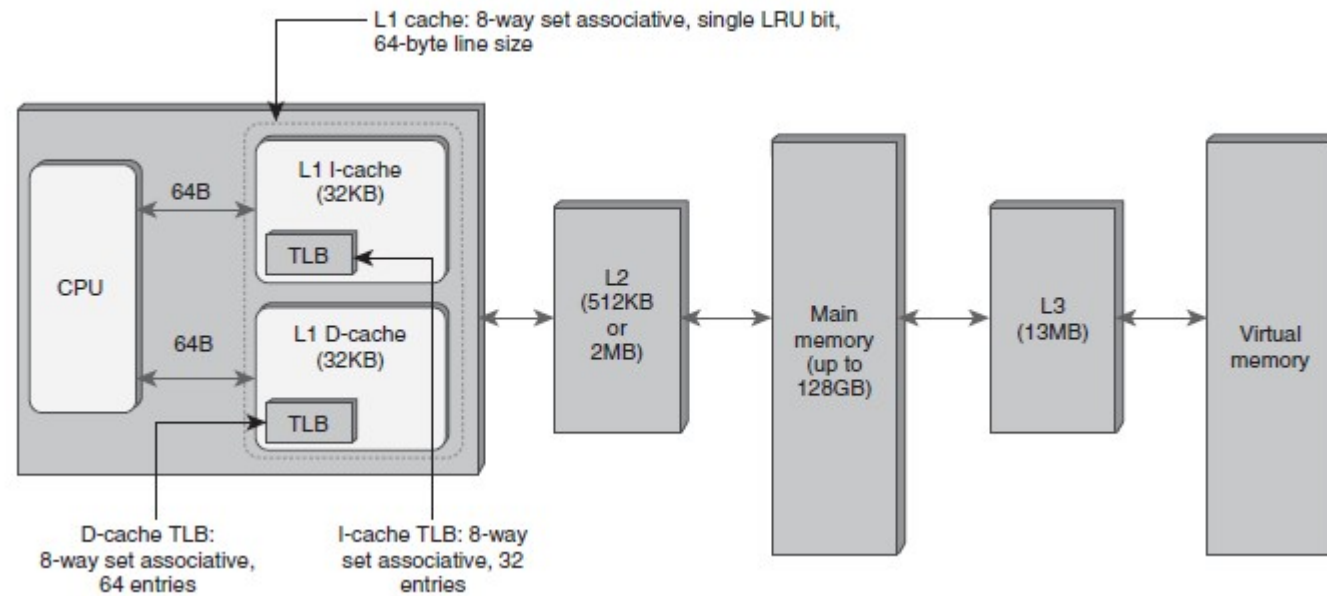
- Large page tables are cumbersome and slow, but with its their uniform memory mapping, page operations are fast. Segmentation allows fast access to the segment table, but segment loading is labor-intensive.
- Paging and segmentation can be combined to take advantage of the best features of both by assigning fixed-size pages within variable-sized segments.
- Each segment has a page table. This means that a memory address will have three fields, one for the segment, another for the page, and a third for the offset.

6.6 A Real-World Example (1 of 2)

- The Intel Core i9 architecture supports both paging and segmentation, and they can be used in various combinations including unpaged unsegmented, segmented unpaged, and unsegmented paged.
- The processor supports two levels of cache (L1, L2, and L3), having various block sizes.
- The L1 instruction cache (I-cache) and data cache (D-cache) are next to the processor; the L2 cache sits between the processor and memory, and the L3 cache sits between main memory and virtual memory.

The next slide shows this organization schematically.

6.6 A Real-World Example (2 of 2)



Conclusion (1 of 2)

- Computer memory is organized in a hierarchy, with the smallest, fastest memory at the top and the largest, slowest memory at the bottom.
- Cache memory gives faster access to main memory, while virtual memory uses disk storage to give the illusion of having a large main memory.
- Cache maps blocks of main memory to blocks of cache memory. Virtual memory maps page frames to virtual pages.
- There are three general types of cache: direct-mapped, fully associative, and set-associative.

Conclusion (2 of 2)

- With fully associative and set-associative cache, as well as with virtual memory, replacement policies must be established.
- Replacement policies include LRU, FIFO, or LFU. These policies must also take into account what to do with dirty blocks.
- All virtual memory must deal with fragmentation, internal for paged memory, and external for segmented memory.